

Examen Técnico – Desarrollador Backend SR (Java / Spring Boot)

Objetivo del examen:

Evaluar la capacidad para diseñar, desarrollar e implementar una API REST robusta, siguiendo buenas prácticas de desarrollo, principios SOLID, uso adecuado de Spring Boot y manejo de persistencia.

Instrucciones:

- Tiempo estimado: 3 horas
- Puede usar cualquier IDE, pero debe subir el código en un repositorio (GitHub).
- Debe incluir instrucciones claras de cómo ejecutar el proyecto (README).
- Usar Java 11 o superior, Spring Boot 2.7+ o 3.x.
- Base de datos relacional (MySQL, PostgreSQL, etc.).
- El código debe incluir pruebas unitarias.
- Seguridad con Spring Security (JWT básico).
- Contenedor Docker para correr la app.
- Logging estructurado y manejo de errores detallado.

Caso práctico: API de gestión de órdenes de transporte

Desarrolla una API REST para la gestión de **órdenes de transporte** de una empresa de movilidad.

Entidades principales:

1. Order

- id: UUID
- status: ["CREATED", "IN_TRANSIT", "DELIVERED", "CANCELLED"]
- origin: String
- destination: String
- createdAt: DateTime
- updatedAt: DateTime

2. Driver

- id: UUID
- name: String
- licenseNumber: String
- active: boolean

Requerimientos funcionales:

1. Gestión de órdenes:

- Crear una nueva orden.
- Cambiar el estado de la orden (validar flujo válido entre estados).
- Consultar orden por ID.
- Listar órdenes con filtros: por status, fecha, origen o destino.

2. Gestión de conductores:

- Crear un conductor.
- Listar todos los conductores activos.

3. Asignación de Ordenes

- Asignar un conductor a una orden (solo si el conductor está activo y la orden está en estado CREATED).
- Agregar un archivo a la asignación (PDF)
- Agregar una imagen (.png, .jpg)

Requerimientos técnicos:

- Estructura modular y limpia del proyecto.
- Uso de anotaciones estándar de Spring (@RestController, @Service, @Repository, etc.).
- Validaciones a nivel DTO con @Valid y javax.validation.constraints.
- Manejo adecuado de excepciones con @ControllerAdvice.
- Documentación de la API con Swagger/OpenAPI.
- Pruebas unitarias con JUnit 5 y Mockito.
- Uso de Git (historial de commits ordenado y significativo).
- Uso de MapStruct o manual mapping entre entidades y DTOs (deseable).